



CSE366: Artificial Intelligence (Section-09)
Summer 2026

Project Report

Othello AI Game

<https://github.com/Taoshif1/Othello-AI-GAME>

Submitted by:

Student ID	Student Name	Contribution Percentage
2023-3-60-407	Azizul hakim Siam	35 %
2023-3-60-376	Gazi Taoshif	25%
2023-3-60-417	Aysha Samad Ria	20%
2023-3-60-354	Maria Afroz	20%

Abstract

Our project presents an AI-based implementation of a popular game named Othello (Reversi). The game was built in Python using the Pygame library. The addressed problem is enabling a computer opponent to play a two-player, zero-sum board game competitively against a human. Three adversarial search strategies were implemented and compared. The strategies were:

1. Random Move Selection as Easy mode,
2. Minimax search to depth 2 as medium mode.
3. Minimax with Alpha-Beta Pruning to depth 3 as Hard mode

Move quality is guided by a heuristic evaluation function that combines piece-count difference, corner occupancy, edge occupancy, and mobility. A coin-toss mechanism determines which side the human and the AI control at the start of each match. Also a live statistics panel reports how many nodes were searched, branches pruned, the evaluation score, and the thinking time it takes for AI to make a move. The key finding was that the Alpha-Beta Pruning algorithm reaches the same move decisions as pure Minimax while exploring fewer nodes which confirms the theoretical efficiency gain of pruning. The project concludes that even a modestly deep, heuristic-guided adversarial search produces a noticeably stronger and more strategically sound opponent than random play.

Introduction

Problem Definition

Othello is a two-player, perfect-information, zero-sum strategy game. It is played on a grid of Black and White discs, where a legal move must trap one or more opposing discs between the newly placed disc and another disc of the same colour along a straight line; all trapped discs are then flipped to the mover's colour. The problem this project addresses is designing a computer agent that can choose competitive moves in this environment without knowing surely the outcome of the game in advance. This is challenging

because the number of possible board states grows gradually with each play, so an agent cannot simply search to the end of the game within a reasonable time budget, it must instead search a limited number of moves ahead and decide how good a resultant position is.

Project Goal and Scope

The prime goal of the project is to implement and compare the three game-playing strategies which are Random Move Selection, Minimax, and Minimax with Alpha-Beta Pruning and to measure the difference between them using search statistics such as nodes explored, pruning count, and execution time. A secondary goal is to build a complete, playable Human vs AI application around these algorithms . The scope is on purpose limited to a 6×6 board rather than the standard 8×8 board to keep the Minimax and Alpha-Beta search trees shallow enough to run in real time without additional optimizations such as iterative deepening or transposition tables. The project also supports Human vs AI play only, AI-vs-AI and online multiplayer modes are outside the current scope.

Theoretical Background

Core Concept

The project uses the concept of adversarial search such as Minimax and the optimized version of Minimax with Alpha Beta Pruning. Minimax is an adversarial search algorithm for two-player, zero-sum games. It assumes the AI player always chooses the move that maximizes evaluation score, while the opponent is assumed to always choose the move that minimizes that same score. The algorithm recursively expands the game tree alternating between maximizing and minimizing levels until a fixed depth limit is reached or the game ends, at which point a static evaluation function estimates the value of the resulting board.

Alpha-Beta Pruning is the optimized version of Minimax that produces the matching move decision while visiting far fewer nodes. It maintains two bounds while searching. One is alpha which is the best score the maximizing player can already guarantee, and the other is beta the best score the minimizing player can already guarantee. The branches that cannot effect the final decisions gets pruned and a branch is selected when $\beta \leq \alpha$.

The board evaluation function used to score non-terminal states combines four heuristics into a single numeric value:

Evaluation Score = Piece Score + Corner Score + Edge Score + Mobility Score

1. Piece Score = the AI's disc count minus the opponent's disc count.
2. Corner Score = ± 25 points per corner square (0,0 / 0,COLS-1 / ROWS-1,0 / ROWS-1,COLS-1) held by the AI or opponent, since a corner disc can never be flipped.
3. Edge Score = ± 3 points per non-corner edge square held by the AI or opponent, since edge discs are comparatively hard to flip.
4. Mobility Score = (number of AI legal moves – number of opponent legal moves) $\times 2$, rewarding positions that keep more options open for the AI while restricting the opponent.

Implementation Details

The board is represented as a 6 $\times$ 6 nested Python list with each cell holding one of three integer constants: EMPTY (0), BLACK (1), or WHITE (-1) encoding the two players as +1/-1 lets the same evaluation and search code run for either colour by simply opposing the player value. Legal-move generation walks outward from each empty cell along eight direction vectors (up, down, left, right and the four diagonals) to test whether a straight line of opponent discs is bounded by a disc of the mover's colour. The recursive search itself is implemented with plain function calls rather than an explicit stack, with each recursive branch operating on a deep copy of the board so

that exploring one line of play never disturbs another. A separate module `ai_stats.py` accumulates per-move statistics — nodes visited, branches pruned, evaluation score, algorithm name, and elapsed time — which the UI reads after every AI move.

Methodology and Implementation

Environment Setup

As this project is an implementation of a search based algorithm and has no relation to machine-learning, so its “environment” is the Othello game itself. The playing surface is a 6×6 grid consisting 36 cells whereas the traditional game uses a 8x8 board , which keeps the branching factor and search-tree size small enough for Minimax and Alpha-Beta to run within a fraction of a second at the configured depths. A game state is fully described by the 6×6 board array together with whose turn it is; the “cost function” that guides search is the heuristic evaluation function described before, and for Alpha-Beta specifically, the search additionally tracks the alpha and beta bounds described above to decide when a branch can be safely skipped.

Algorithm Design

State representation: each state is the 6×6 board array plus the player to move. BLACK is +1 and WHITE is −1.

Transition function: a move is a row and column pair, applying a move places a disc of the mover's colour, then scans all eight directions from that cell and flips every contiguous run of opponent discs that is terminated by another disc of the mover's colour.

Goal test: the game ends when neither player has any legal move remaining on the current board; if only one side has no legal move, that side's turn is skipped and play passes to the opponent instead of ending the game.

Difficulty selection maps directly onto three implementations of the search:

Difficulty	Algorithm	Search Depth	Pruning
Easy	Random Move Selection	N/A	No
Medium	Minimax	2	No
Hard	Minimax + Alpha-Beta	3	Yes

For Medium and Hard, `get_best_move()` function tries every legal move for the AI, applies it to a copied board, and recursively scores the resulting position with `minimax()` or `alpha_beta()` with alpha and beta initialized to $-\infty$ and $+\infty$; the move with the highest returned score is chosen. Easy mode instead calls `random_move()`, which simply picks uniformly at random among the legal moves.

Implementation Language and Tools

1. Language: Python 3
2. Library: Pygame for rendering the board, menus, coin-toss screen, statistics panel, and end screen, and for handling mouse or click input
3. Standard library: `random` for random move selection and coin toss , `math` for $\pm$ infinity bounds for Minimax/Alpha-Beta , `time` for measuring AI thinking time.

Results and Analysis

Experimental Setup

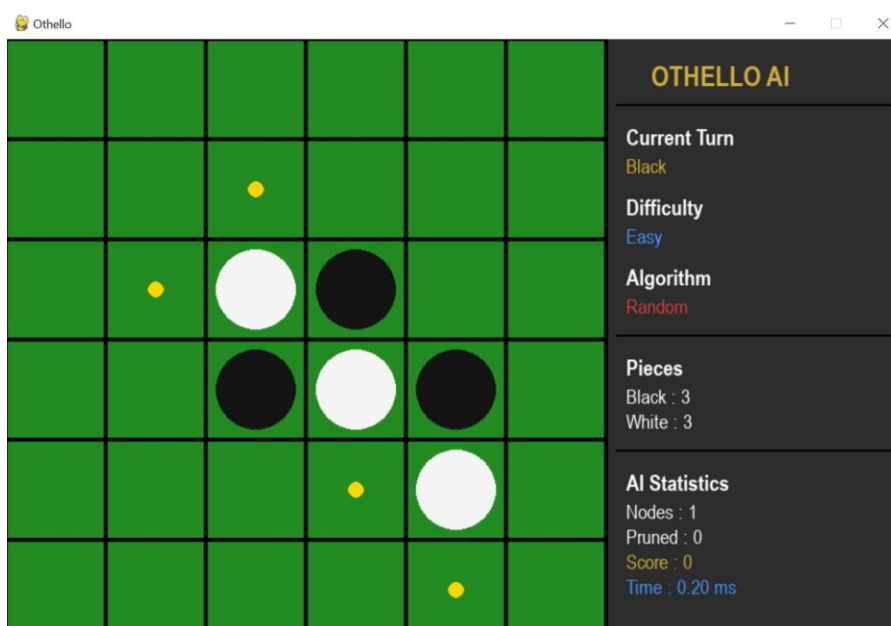
This application reports four metrics after every AI move captured directly from the search:

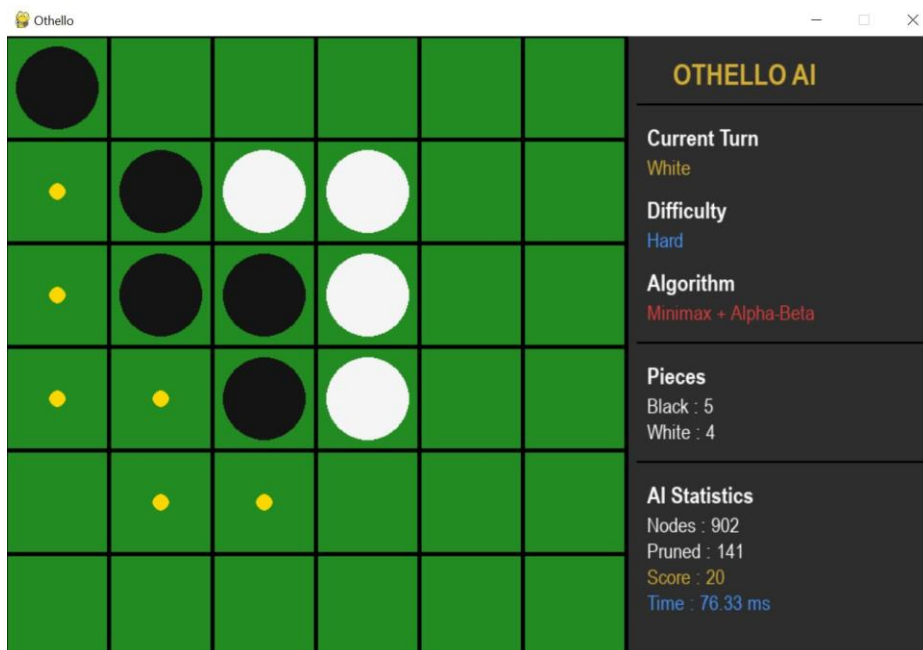
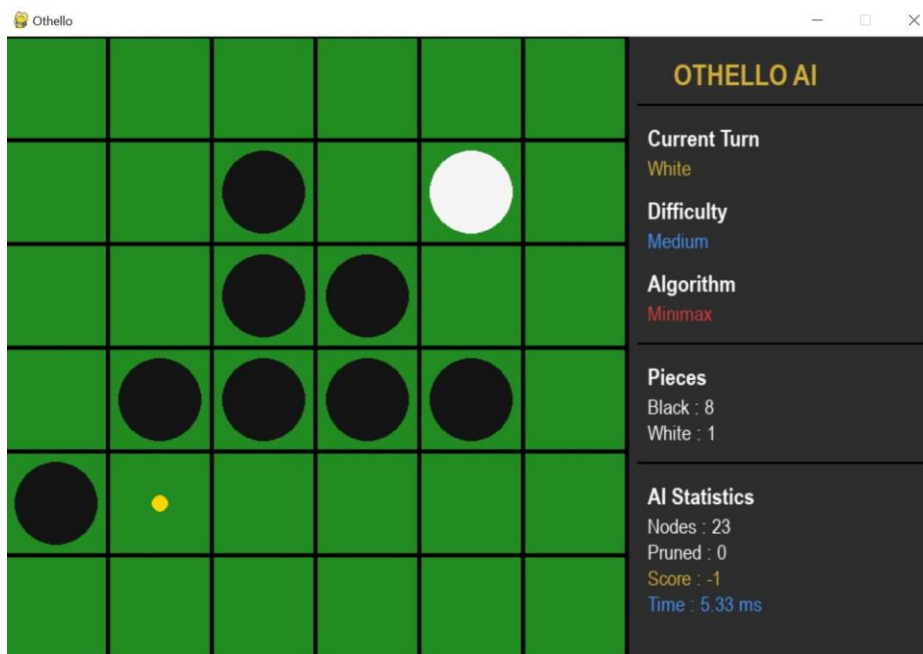
1. the algorithm name
2. the number of nodes searched
3. the number of pruning events
4. the resulting evaluation score
5. and the AI's execution time in milliseconds.

Because Medium mode uses pure Minimax, no pruning happens during so its pruning count should remain 0 for every move, which is a useful built-in check that Alpha-Beta specific code is not accidentally running in Medium mode. Experiments can be conducted by playing repeated matches at each difficulty and recording these statistics move-by-move or averaging them over a full game, and by playing many games per difficulty to estimate an AI win rate against a human or against another difficulty level.

Results Presentation

A sample statistics readout captured after a single Hard-mode move illustrates the kind of data the panel exposes:





Discussion and Interpretation

The three difficulty levels form a clear progression in decision quality. Random Move Selection has no look ahead at all, so it plays legally but with no regard for corners, edges, or mobility; it exists mainly as a low baseline for comparison. Minimax at depth 2 evaluates the immediate consequences of a move and the opponent's best reply, which is enough to avoid obviously bad

moves such as handing over a corner but is still short-sighted relative to the optimized version. Minimax with Alpha-Beta Pruning searches to depth 3 while returning exactly the same move choice that unpruned Minimax would return at that depth. Pruning only removes branches that provably cannot affect the result so the advantage it provides is efficiency rather than move quality. It can look twice as deep as Medium mode in the same time because it explores far fewer nodes per level.

The main strength of the implementation is that its heuristic explicitly encodes well-known Othello strategy like corner and edge stability, and mobility rather than relying on piece count alone, which by itself is a weak signal in the early and middle game. Its main weakness is the fixed, hand-tuned weighting of the four heuristic terms which are ± 25 for corners, ± 3 for edges and $\times 2$ for mobility difference. This was not learned or tuned against real game data and may not be optimal, especially as the 6×6 board's dynamics differ somewhat from the standard 8×8 board most published Othello heuristics assume.

Error analysis: because the search depth is fixed and not adaptive, the AI can occasionally misjudge sharp tactical positions near the end of the game, where a full-depth search to the actual end of the game (only feasible on the smaller 6×6 board) would be more accurate than the current fixed 2- or 3-ply lookahead.

Conclusion

Our project implemented and compared three adversarial search strategies — Random Move Selection, Minimax, and Minimax with Alpha-Beta Pruning — inside a complete, playable Othello application. The original goal, stated in Section 3.2, of implementing and quantitatively comparing these strategies using search statistics such as nodes explored and pruning counts, was achieved: the built-in AI Statistics panel exposes exactly these numbers after every AI move, and the difficulty levels map directly onto the three algorithms. The single most important takeaway is that Alpha-Beta Pruning delivers the same decision quality as plain Minimax at a fraction of the search

cost, which in practice is what allows the Hard difficulty to search twice as deep (3 ply) as Medium (2 ply) within a comparable time budget — a direct, hands-on demonstration of why pruning matters in adversarial search.

8. Future Work

1. Extend the board from 6×6 to the standard 8×8 Othello board.
2. Make the Minimax/Alpha-Beta search depth adjustable rather than fixed.
3. Add more advanced heuristics, such as stable-piece and frontier-piece evaluation, and dynamic evaluation weights.
4. Add iterative deepening and transposition tables to search deeper within the same time budget.
5. Add an AI-vs-AI mode to let the three difficulty levels play each other automatically and collect win-rate statistics.
6. Add move history, individual player timers, save/load, sound effects, piece-flipping animation, and online multiplayer.

References (APA Style)

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson. [Minimax search and Alpha-Beta Pruning, Chapter 5: Adversarial Search and Games]

Pygame Community. (n.d.). Pygame documentation. Retrieved from <https://www.pygame.org/docs/>